

GeoSlices - Calcular áreas em mapas

Introdução ao projeto

O GeoSlices é uma aplicação web com a proposta de permitir o cálculo de área de terrenos em formatos regulares ou irregulares, utilizando métodos comuns, soma de riemann, e métodos avançados.

Ferramentas

HTML: Linguagem de marcação base para toda ferramenta web

CSS: Linguagem que define a aparência e estilo das páginas web

Javascript: Linguagem de programação utilizada na web, adiciona funcionalidades e dinâmica às páginas.

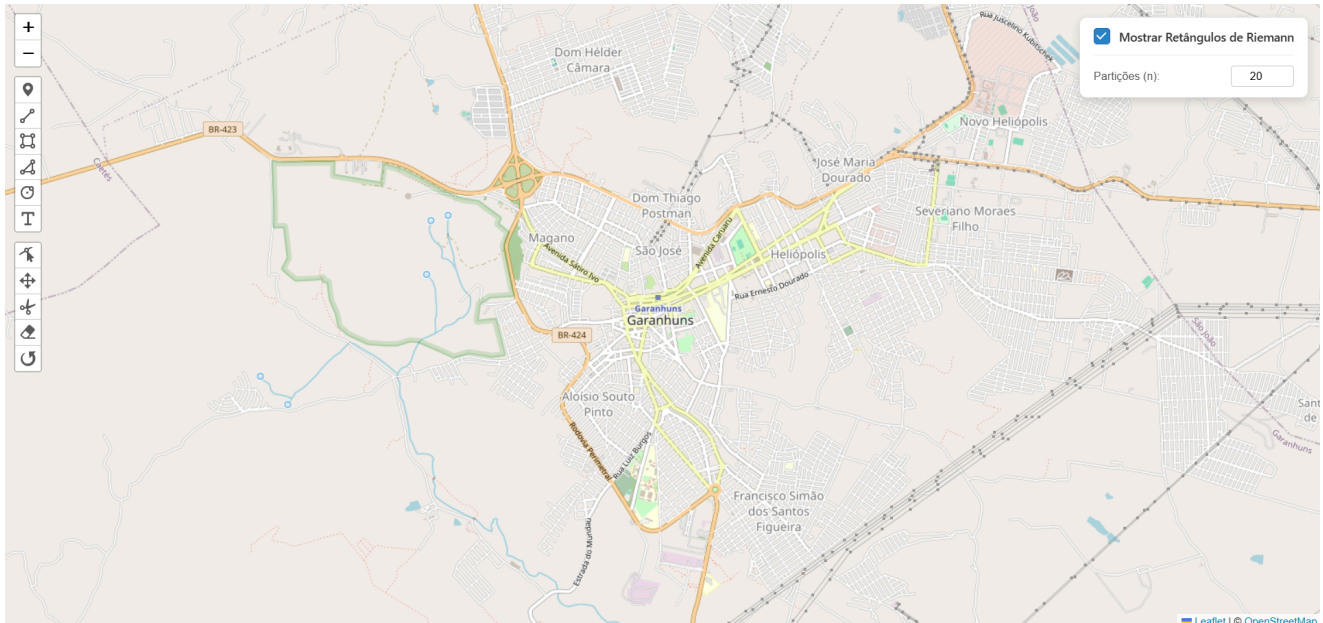
Leaflet: Biblioteca utilizada para geração de mapas interativos

Geoman: Biblioteca que permite desenhar sobre mapas do Leaflet

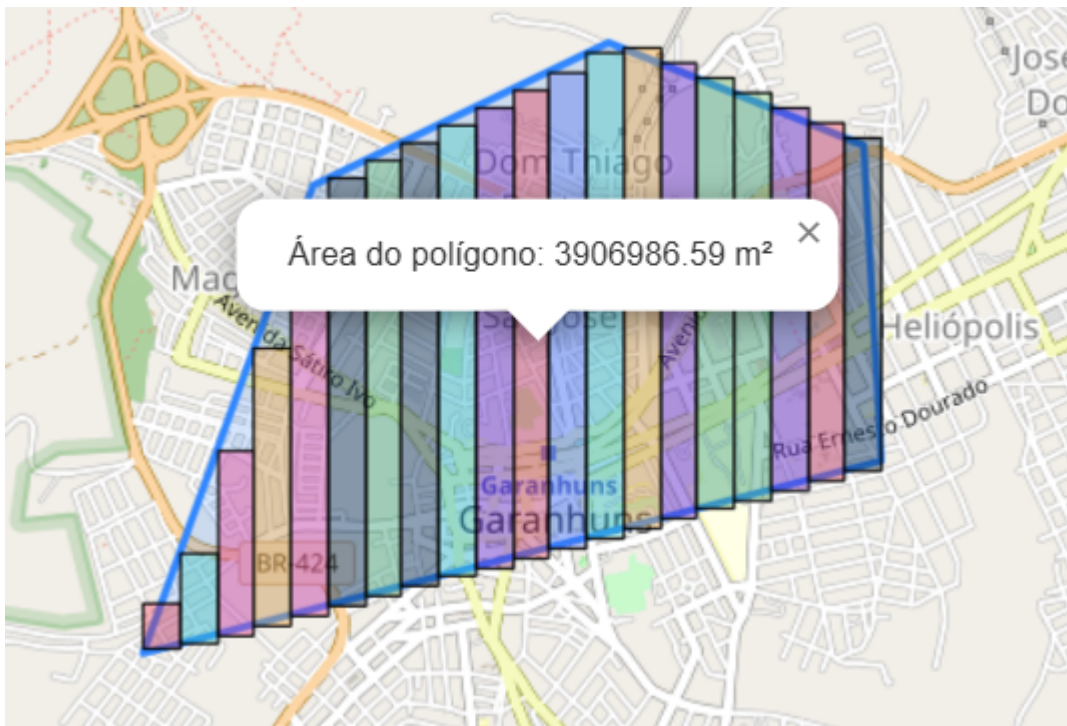
Turf: Biblioteca para análise espacial e operações geométricas

Visão geral

Tela inicial



Calculando áreas





Funcionalidades

Para o funcionamento do site, é necessário importar as bibliotecas Leaflet, Geoman e Turf, como mostrado no trecho de código abaixo

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<!--
```

Leaflet: Apresentar o mapa

Geoman: Desenhar no mapa

Turf: Verificar polígonos e calcular áreas

```
-->
```

```
<link
```

```
rel="stylesheet"
```

```
href="https://unpkg.com/@geoman-io/leaflet-geoman-free@latest/dist/leaflet-geoman.css">
```

```
<link rel="stylesheet"
```

```

href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"

integrity="sha256-p4NxAoJBhIIN+hmNHrzRCf9tD/miZyoHS5obTRR9BMY="

crossorigin="" />

<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"

integrity="sha256-20nQCchB9co0qIjJZRGuk2/Z9VM+kNiyxNV1vTlZBo="

crossorigin=""></script>

<script src="https://unpkg.com/@geoman-io/leaflet-geoman-
free@latest/dist/leaflet-geoman.js"></script>

<script src="https://cdn.jsdelivr.net/npm/@turf/turf@7/turf.min.js">
</script>

<title>GeoSlices - Cálculo de área</title>

```

Apresentação de mapa

Os trechos abaixo definem um mapa do Leaflet com as coordenadas de garanhuns, e o adicionam ao site

```

<div id="map">
</div>

```

```

coordenadasGaranhuns = [-8.890278, -36.492778]

```

```

//Definir a coordenada e zoom inicial

```

```

const map = L.map('map', {pmIgnore: false}).setView(coordenadasGaranhuns,
14);

```

```

L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', {maxZoom: 18,
attribution: '&copy; <a
href="http://www.openstreetmap.org/copyright">OpenStreetMap</a>'
}).addTo(map);

```

Desenho sobre o mapa

Para essa função, é utilizada a biblioteca Geoman, que adiciona uma barra de ferramentas para desenhar e manipular desenhos.

Para adicionar essa barra, é utilizado o seguinte trecho:

```
//Adicionar a barra de ferramentas de desenho
map.pm.addControls({

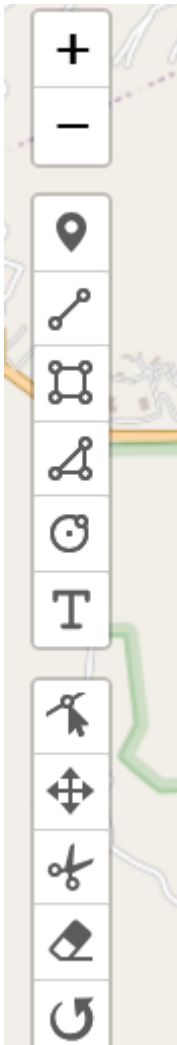
  position: 'topleft',

  drawCircleMarker: false,

  rotateMode: true,

});
```

Gerando uma barra no canto superior esquerdo da tela, como mostra a figura abaixo



As ferramentas listadas são, respectivamente:

1. Aumentar zoom
2. Diminuir zoom
3. Adicionar marcador

4. Desenhar livremente a partir de pontos
5. Desenhar retângulos
6. Desenhar polígonos
7. Desenhar Círculos
8. Adicionar texto
9. Manipular pontos
10. Mover figura
11. Cortar figura
12. Apagar figura
13. e Rotacionar figura

Cálculo de área

A partir dos desenhos gerados ou editados, o site calcula a área da figura de acordo com o seu tipo, podendo ele ser um retângulo, um círculo, um polígono irregular elegível ou não para a soma de Riemann.

Para isso, definimos a função "calcularArea", que recebe uma figura e verifica em qual tipo ela se encaixa.

O site permite apenas um desenho por vez, apagando automaticamente o ultimo desenho assim que um novo é criado. A área do desenho é recalculada sempre que um novo desenho é criado, ou o desenho atual é editado. Esse comportamento é mostrado no trecho de código abaixo:

```
//Verificar o desenho atual
map.on('pm:create', function(e){
  const layer = e.layer;
  const shape = e.shape;

  if(desenhoAtual !== null){
    map.removeLayer(desenhoAtual);
  }

  desenhoAtual = layer;
  calcularArea(layer, shape)
  layer.on('pm:edit', function(){
    calcularArea(layer, shape)
  })
})

map.on('pm:remove', function(e){

  if(desenhoAtual === e.layer){
    desenhoAtual = null
```

```
}
})
```

Retângulo

Para calcular a área de um retângulo, utilizamos a função `calcularArea` nele, verificando a latitude e longitude dos seus vértices, convertendo as distâncias entre eles para metros e, a partir disso, calculando a base e a altura. Após isso, aplica-se a fórmula *base * altura*, como mostrado no trecho abaixo

```
function calcularArea(layer, shape){

  if (shape == 'Rectangle'){ //caso seja um retângulo
    let vertices = layer.getLatLngs()[0]
    let ponto1 = vertices[0]
    let ponto2 = vertices[1]
    let ponto3 = vertices[2]

    let base = map.distance(ponto1, ponto2)
    let altura = map.distance(ponto2, ponto3)

    let area = base*altura

    const texto = `A área do retângulo é ${area.toFixed(2)} m²`
    layer.bindPopup(texto).openPopup();

  }
}
```

A função `layer.bindPopup` permite adicionar um texto que acompanhe o desenho, permitindo que a área esteja sempre próxima à figura desenhada.

Círculo

Para calcular a área do círculo, se recebe o raio a partir da função "getRadius", se executa a fórmula $\pi * raio^2$ utilizando a biblioteca `Math`.

Para exibir essa área, utiliza-se a mesma lógica presente no cálculo anterior, como mostra o trecho de código

```
if(shape=='Circle'){
  let raio = layer.getRadius();
  let area = Math.PI * Math.pow(raio, 2)
  const texto = `A área do círculo é ${area.toFixed(2)} m²`
}
```



```
layer.bindPopup(texto).openPopup();
}
```

Polígono

Para o cálculo de polígonos, primeiro, é verificado se é possível calcular sua área utilizando a soma de Riemann, isso se faz utilizando a função "unkinkPolygon" da biblioteca Turf, que divide o polígono principal em polígonos menores caso ele possua linhas que se cruzam. Após isso, é verificado se essa função gerou mais de um polígono, se sim, é indicado que não é possível calcular sua área a partir da soma de Riemann, caso contrário, é possível. Essa verificação é adicionada a uma função chamada "verificarPoligono", mostrada no trecho abaixo:

```
function verificarPoligono(layer){
  let coordenadas = layer.getLatLngs()[0]
  let gj = layer.toGeoJSON()

  const poligonos = turf.unkinkPolygon(gj)

  if(poligonos.features.length > 1){

    console.log('O polígono possui linhas que se cruzam')

    return false;

  }
  return true;
}
```

As figuras abaixo apresentam exemplos

Polígono válido



Polígono inválido:



Soma de Riemann

O cálculo da soma de Riemann está dividido em duas funções, na função `calcularArea`, possuímos todos os passos para definir o valor `a`, `b`, `n`, e largura dos retângulos.

Na função `calcularAlturasFatia`, a partir da posição do retângulo e do desenho do polígono, se define qual altura o retângulo deve ter.

```
function calcularAlturasFatia(coordenadas, lngAtual) {
  let pontosDeCorte = [];
  // Varre todas as arestas do polígono
  for (let j = 0; j < coordenadas.length; j++) {

    let p1 = coordenadas[j];

    let p2 = coordenadas[(j + 1) % coordenadas.length]; // Fecha o circuito com
    o próximo ponto
  }
}
```

```
// Verifica se a longitude atual passa entre p1 e p2

if ((p1.lng <= lngAtual && p2.lng > lngAtual) || (p2.lng <= lngAtual &&
p1.lng > lngAtual)) {

// Interpolação linear para achar a latitude exata do corte

let t = (lngAtual - p1.lng) / (p2.lng - p1.lng);

let latCorte = p1.lat + t * (p2.lat - p1.lat);

pontosDeCorte.push(latCorte);

}

}

// Se cruzou o polígono corretamente (achou teto e piso), retorna os valores
if (pontosDeCorte.length >= 2) {

return {

tetoLat: Math.max( ... pontosDeCorte),

pisoLat: Math.min( ... pontosDeCorte)

};

}

// Se não cruzou o polígono nessa longitude, retorna null

return null;

}
```

A função `calcularAlturasFatia` é ativada para cada retângulo do cálculo, o somatório da área desses retângulos é feito

```
let coordenadas = layer.getLatLngs()[0];

let minLng = Math.min( ... coordenadas.map(p => p.lng));
```

```

let maxLng = Math.max( ... coordenadas.map(p => p.lng));

let n = parseInt(document.getElementById('txt-n-riemann').value);

// Calcular larguras/deltas dos retângulos

let deltaLng = (maxLng - minLng) / n;

let menorLatGlobal = Math.min( ... coordenadas.map(p => p.lat));

let deltaMetros = map.distance(L.latLng(menorLatGlobal, minLng),
L.latLng(menorLatGlobal, minLng + deltaLng));

// Limpar desenho de retângulos anterior

camadaRetangulos.clearLayers();

// Soma de riemann

for (let i = 0; i < n; i++) {

    let lngAtual = minLng + i * deltaLng;

    let lngProxima = lngAtual + deltaLng;

    let lngMedio = lngAtual + deltaLng/2

    let lngAnalise = (i==0)?lngMedio:lngAtual //Impedir erro no índice 0

    let alturas = calcularAlturasFatia(coordenadas, lngAnalise); //
    Calcular a altura dos retangulos

    if (alturas !== null) {

        let tetoLat = alturas.tetoLat;

        let pisoLat = alturas.pisoLat;

        let alturaMetros = map.distance(L.latLng(tetoLat, lngAnalise),

```

```
L.latLng(pisoLat, lngAnalise));

    areaTotal += deltaMetros * alturaMetros;
```

e por fim, cada retângulo é adicionado com uma cor aleatória a uma camada para visualização posterior.

```
let cantoS = L.latLng(pisoLat, lngAtual);

let cantoN = L.latLng(tetoLat, lngProxima);

let limitesRet = L.latLngBounds(cantoS, cantoN);

let retangulo = L.rectangle(limitesRet, {

    color: '#000',

    weight: 1,

    fillColor: gerarCorAleatoria(),

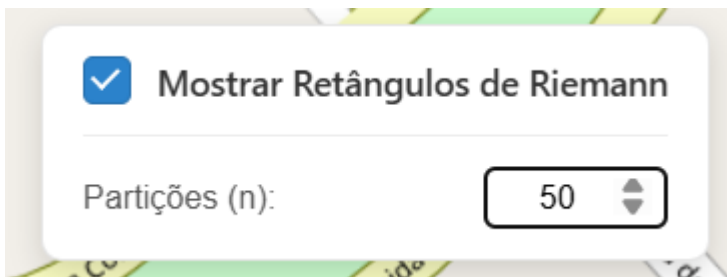
    fillOpacity: 0.35

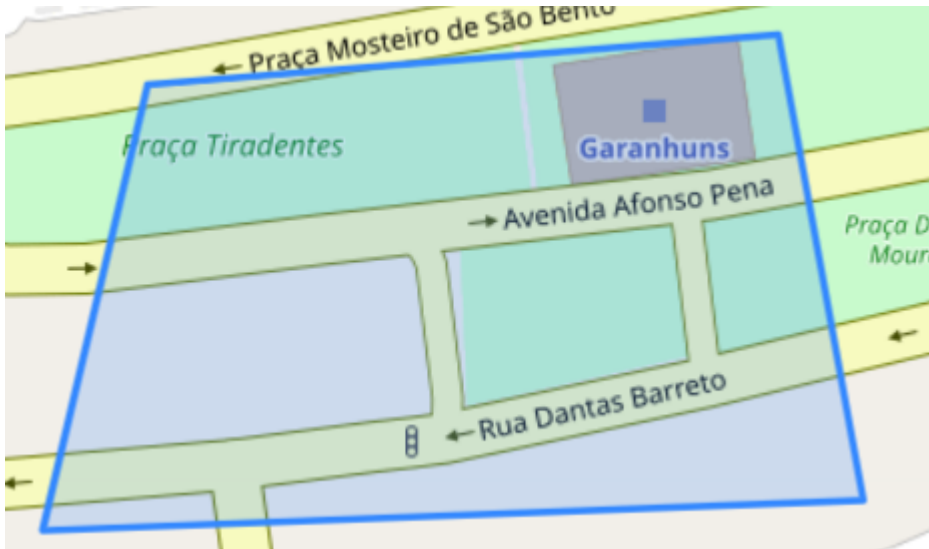
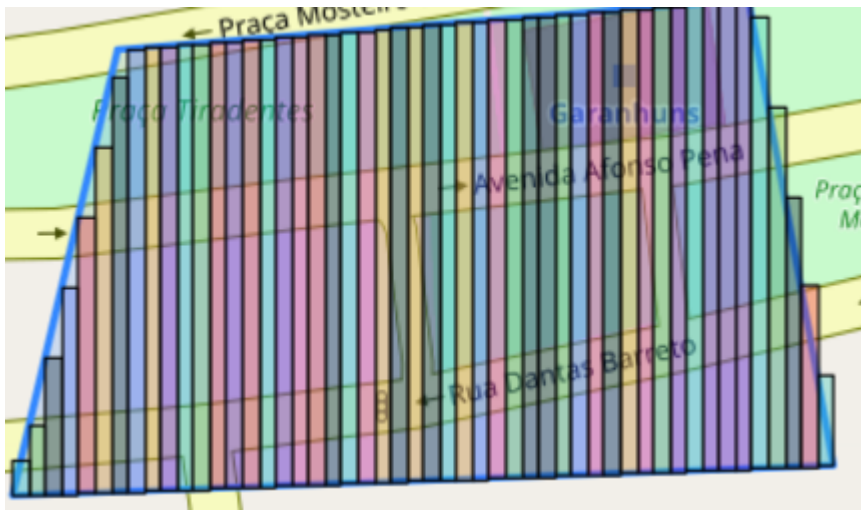
});

camadaRetangulos.addLayer(retangulo);
```

Opções do usuário

Ao usuário, são oferecidas ferramentas para controlar o cálculo e visualização da soma de Riemann, podendo escolher o valor de n e também se os retângulos vão ser visíveis ou não.





Turf Area

Para os casos em que a soma de Riemann não é aplicável, utilizamos a função "area" da biblioteca Turf para não limitar os cálculos do usuário.

O trecho de código e imagem abaixo mostram essa função

```
areaTotal = turf.area(layer.toGeoJSON())
```



Validação

Para validar o cálculo de área do nosso software, utilizamos a ferramenta de cálculo de área da biblioteca Turf, fazendo com que, sempre que uma área seja calculada a partir do desenho do usuário, o *console* exiba também a área calculada pelo Turf e o erro percentual calculado a partir da diferença entre ambas.



Área exata: 8001.598655837094
Margem de erro: 0.031887381727972744%

Limitações

Nosso software nem sempre é o mais exato no cálculo de áreas, após validações utilizando o Turf, notamos que mesmo nas áreas mais simples, há uma margem de erro, seguem as justificativas.

Área do retângulo: A diferença se deve ao modelo utilizado para o cálculo, no nosso projeto, foi utilizada a geometria pura, já o Turf oferece uma versão mais exata considerando o formato esférico da terra. Apesar disso, é uma margem de erro pequena, como mostrado na validação.

Área do círculo: Nosso projeto oferece uma área mais próxima da realidade, devido ao Turf utilizar polígonos que simulam um círculo, em vez de um círculo real, apesar disso, usar um número alto de lados para esse polígono faz com que o erro seja mínimo.

Área de polígonos irregulares: Há sempre uma margem de erro com relação à área real, o cálculo do Turf oferece uma versão mais exata por utilizar formas mais robustas de cálculo, em comparação com a soma de Riemann padrão.

Ferramenta de corte: A utilização da ferramenta de corte não recalcula a área do polígono, gerando uma figura que não corresponde à área apresentada.

Próximos passos

Entre os próximos passos, podem ser listados:

1. Melhorar os métodos de cálculo a fim de reduzir essa margem de erro
2. Remover falhas como a da ferramenta de corte

3. Adicionar meios para mitigar erros humanos, permitindo um desenho mais exato sobre o terreno que se deseja calcular